

Software engineering

Working with existing code

Skill progressions by applying material:

Code quality +1-4

Communication skill +1



Clean code

Intention revealing names

```
int d; //elapsed time in days
```

```
int days;
```

```
int elapsedTimeInDays;
```

```
int daysSinceCreation;
```

```
int daysSinceModification;
```

```
int fileAgeInDays;
```

```
string[] GetThem(string[] theList)
{
    List<string> list1 = new List<string>();
    for (int i = 0; i < theList.Length; i++)
    {
        if (theList[i][0] == 5)
        {
            list1.Add(theList[i]);
        }
    }
    return list1.ToArray();
}
```

```
string[] GetFlaggedCells(string[] gameBoard)
{
    List<string> flaggedCells = new List<string>();
    for (int i = 0; i < gameBoard.Length; i++)
    {
        if (gameBoard[i][STATUS_INDEX] == FLAGGED)
        {
            flaggedCells.Add(gameBoard[i]);
        }
    }
    return flaggedCells.ToArray();
}
```

```
static void CpyChr(char[] a1, char[] a2)
{
    for (int i = 0; i < a1.Length; i++)
    {
        a2[i] = a1[i];
    }
}
```

```
static void CopyChars(char[] source, char[] destination)
{
    for (int i = 0; i < source.Length; i++)
    {
        destination[i] = source[i];
    }
}
```

Better names: add semantic meaning and remove abbreviations

```
int Prime(int n)
{
    if(n <= 1)
    {
        return 0;
    }
    for (int i = 0; i < n; i++)
    {
        if (n % i == 0)
        {
            return 0;
        }
    }
    return 1;
}
Console.WriteLine(Prime(5));
```

```
bool IsPrime(int checkedNumber)
{
    if (checkedNumber <= 1)
    {
        return false;
    }
    for (int divisor = 2; divisor < checkedNumber;
        divisor++)
    {
        if (checkedNumber % divisor == 0)
        {
            return false;
        }
    }
    return true;
}
Console.WriteLine(IsPrime(5));
```

```

public double relScore(Movie m1, Movie m2)
{
    const double GW = 0.4;
    const double YW = 0.1;
    const double DW = 0.2;
    const double WW = 0.3;

    double s = 0;
    if (m1.gen == m2.gen)
    {
        s += GW;
    }
    if (m1.yr == m2.yr)
    {
        s += YW;
    }
    if (m1.dir == m2.dir)
    {
        s += DW;
    }

    foreach (string w in m1.writers)
    {
        if (m2.writers.IndexOf(w) != -1)
        {
            s += WW;
            break;
        }
    }
    return s;
}

```

```

public double movieRelationScore(Movie movie1, Movie movie2)
{
    const double GENRE_WEIGHT = 0.4;
    const double YEAR_WEIGHT = 0.1;
    const double DIRECTOR_WEIGHT = 0.2;
    const double WRITER_WEIGHT = 0.3;

    double score = 0;
    if (movie1.genre == movie2.genre)
    {
        score += GENRE_WEIGHT;
    }
    if (movie1.year == movie2.year)
    {
        score += YEAR_WEIGHT;
    }
    if (movie1.director == movie2.director)
    {
        score += DIRECTOR_WEIGHT;
    }

    foreach (string writer in movie1.writers)
    {
        if (movie2.writers.Contains(writer))
        {
            score += WRITER_WEIGHT;
            break;
        }
    }
    return score;
}

```

Magic numbers

```
class PaymentProcessor
{
    public double CalculateTax(double amount)
    {
        return amount * 7 / 100;
    }

    public double ConvertToDecimal(int percentage)
    {
        return percentage / 100;
    }
}
```

```
class Circle
{
    public double GetCircumference(double radius)
    {
        return 2 * 3.14159 * radius;
    }
}
```

```
class Circle
{
    /// <summary>
    /// The ratio between the radius and the
    /// circumference of a circle
    /// </summary>
    private const double PI = 3.14159;

    public double GetCircumference(double radius)
    {
        return 2 * PI * radius;
    }
}
```

```
enum OrderStatus
```

```
{  
    Pending = 0,  
    Shipped = 1,  
    Delivered = 2,  
    Cancelled = 3  
}
```

```
class Order
```

```
{  
    public OrderStatus Status { get; set; }  
}
```

```
Order myOrder = new Order();  
myOrder.Status = OrderStatus.Shipped;
```

```
double trapezoidArea =  
    (input[0] + input[1]) * input[2] / 2;
```

```
double base1 = input[0];  
double base2 = input[1];  
double height = input[2];
```

```
double averageBase = (base1 + base2) / 2;  
double trapezoidArea = averageBase * height;
```

```
// Adding 24 ensures rounding up when using  
integer division
```

```
int boxesRequired = (itemsOrdered + 24) / 25;
```

If shorthand

```
_viewModel.PhoneDefault = user == null ? "" :  
    (string.IsNullOrEmpty(user.PhoneDay) ?  
        (string.IsNullOrEmpty(user.PhoneEvening) ?  
            (string.IsNullOrEmpty(user.Mobile) ? "" :  
                user.Mobile) :  
            user.PhoneEvening) :  
        user.PhoneDay);
```

```
if (user == null)  
{  
    _viewModel.PhoneDefault = string.Empty;  
}  
if (!string.IsNullOrEmpty(user.PhoneDay))  
{  
    _viewModel.PhoneDefault = user.PhoneDay;  
}  
if (!string.IsNullOrEmpty(user.PhoneEvening))  
{  
    _viewModel.PhoneDefault = user.PhoneEvening;  
}  
if (!string.IsNullOrEmpty(user.Mobile))  
{  
    _viewModel.PhoneDefault = user.Mobile;  
}
```

- Classes and objects should have noun or noun phrase names like Customer, WikiPage, Account, and AddressParser
- Methods should have verb or verb phrase names like PostPayment, DeletePage, or Save.
- Collections should be plural like points, customerStatistics
- Raw numbers should not exist outside “Constants” file

- Use well named static factory methods instead of more constructors

```
Complex fulcrumPoint = Complex.FromRealNumber(23.0);
```

Instead of

```
Complex fulcrumPoint = new Complex(23.0);
```

- What does the add function do here? `Date newDate = date.Add(5);`
 - If it changes the date then `AddDaysTo` or `IncreaseByDays`
 - Else maybe `DaysLater` or `DaysSince`

Encapsulate Boundary Conditions

```
if (level + 1 < tags.Length)
{
    parts = new Parse(body, tags, level + 1, offset + endTag);
}
```

```
int nextLevel = level + 1;
if (nextLevel < tags.Length)
{
    parts = new Parse(body, tags, nextLevel, offset + endTag);
}
```

Don't return null

```
List<Employee> employees = getEmployees();  
if (employees != null)  
{  
    foreach (Employee employee in employees)  
    {  
        totalPay += employee.getPay();  
    }  
}
```

```
List<Employee> employees = getEmployees();  
foreach (Employee employee in employees)  
{  
    totalPay += employee.getPay();  
}  
  
public List<Employee> getEmployees()  
{  
    if (/*..there are no employees.. */) {  
        return new List<Employee>();  
    }  
}
```

Comments

```
// Check to see if the employee is eligible for full benefits
```

```
if ((employee.flags & HOURLY_FLAG) && (employee.age > 65))
```

```
if (employee.isEligibleForFullBenefits())
```

```
// format matched hh:mm:ss EEE, MMM dd, yyyy
```

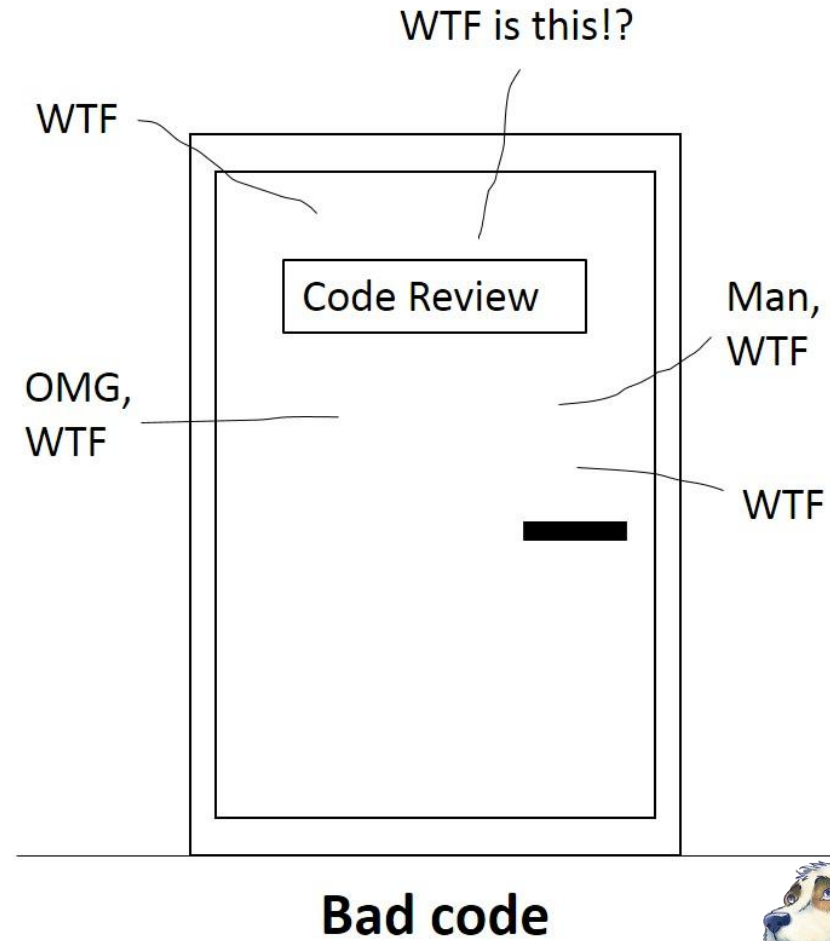
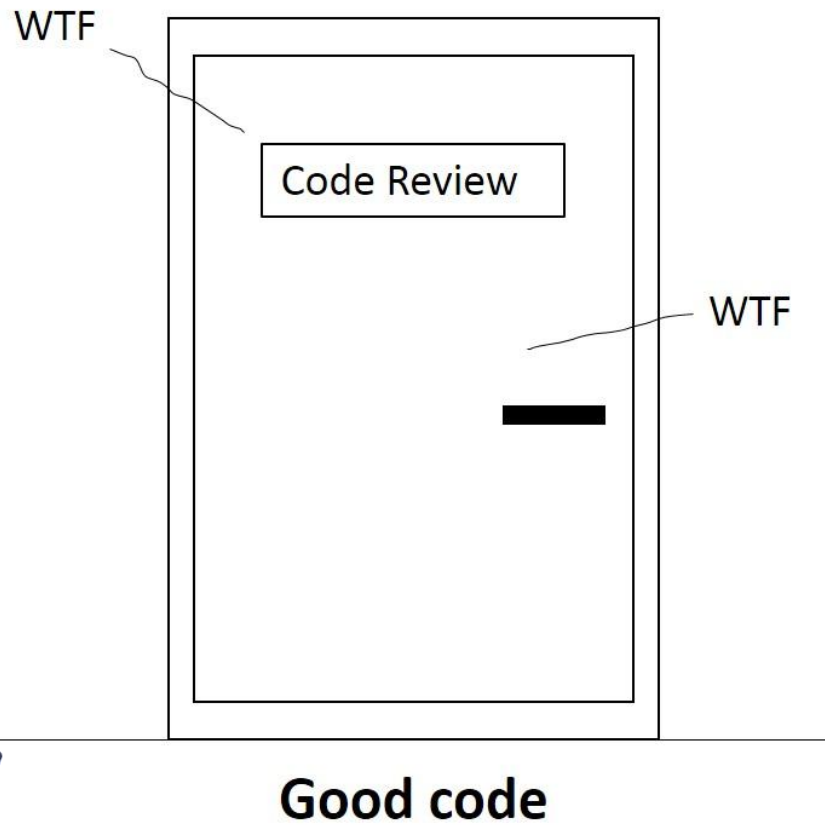
```
Pattern timeMatcher = Pattern.compile("\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

Specifications should have:

- A relevant description
- Type and description of parameters
- Conditions imposed over input (pre-conditions)
- Type and description of return value
- Conditions true after function is done (post-conditions)
- Exceptions thrown
- Now, in practice...

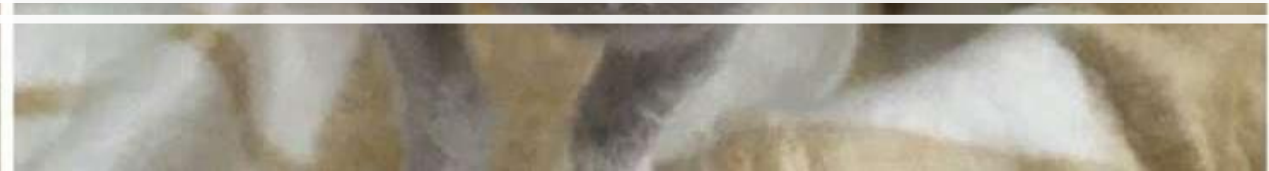
```
/// <summary>
/// Copy characters from source to destination
/// </summary>
/// <param name="source">Non null character array</param>
/// <param name="destination">Non null character array</param>
/// <returns>True if the copy was successful, False otherwise</returns>
/// <exception cref="NullReferenceException">Thrown when source or
/// destination are null</exception>
static bool copyChars(char[] source, char[] destination)
{
    if(source.Length < destination.Length)
    {
        return false;
    }
    for (int i = 0; i < source.Length; i++)
    {
        destination[i] = source[i];
    }
    return true;
}
```

The only valid measurement of code quality: WTFs/minute

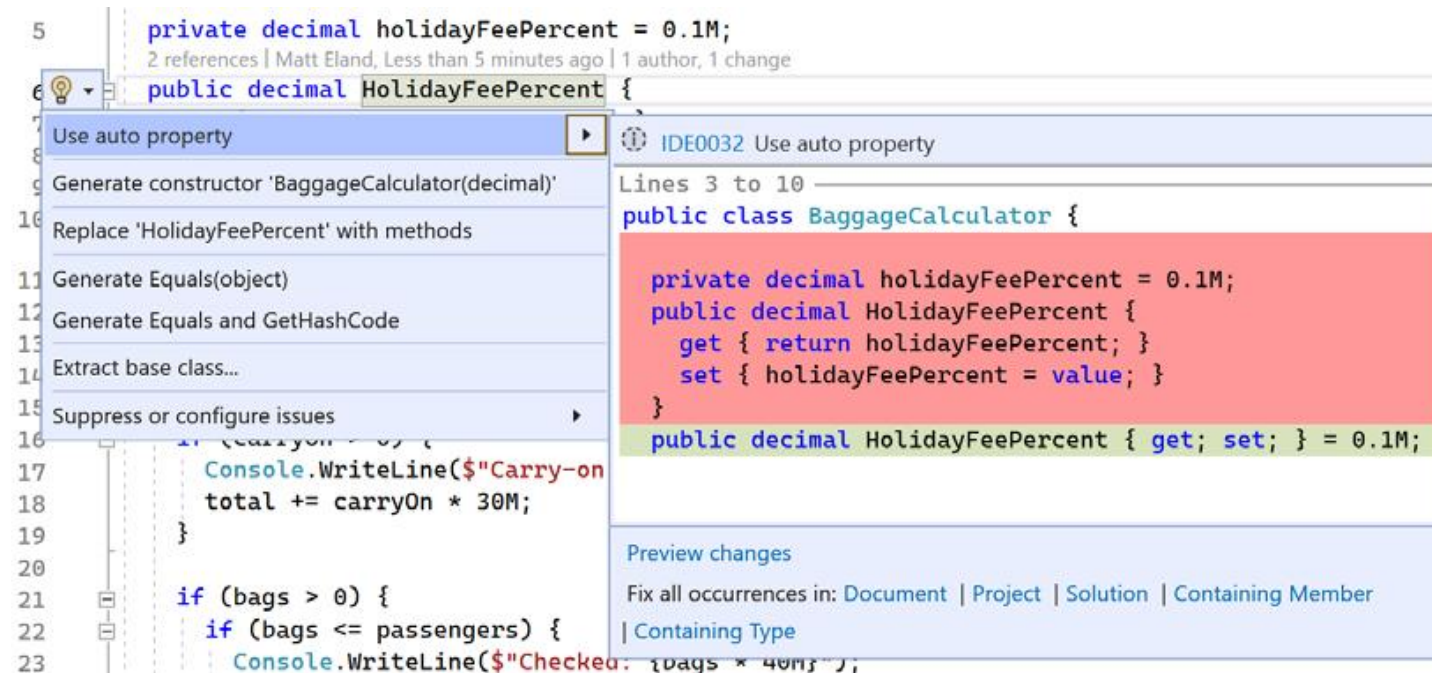




Refactoring



Use auto property



Introduce local

```
void RenderBanner()
{
    if ((platform.ToUpper().IndexOf("MAC") > -1) &&
        (browser.ToUpper().IndexOf("IE") > -1) &&
        wasInitialized() && resize > 0)
    {
        // do something
    }
}
```

Introduce local for '(platform.ToUpper().IndexOf("MAC") > -1)'

Introduce local for all occurrences of '(platform.ToUpper().IndexOf("MAC") > -1)'

Lines 74 to 76

```
{
- if ((platform.ToUpper().IndexOf("MAC") > -1) &&
+ bool v = (platform.ToUpper().IndexOf("MAC") > -1)
+ if (v &&
    (browser.ToUpper().IndexOf("IE") > -1) &&
    wasInitialized() && resize > 0)
{
    // do something
}
```

[Preview changes](#)

```
void RenderBanner()
{
    if ((platform.ToUpper().IndexOf("MAC") > -1) &&
        (browser.ToUpper().IndexOf("IE") > -1) &&
        wasInitialized() && resize > 0)
    {
        // do something
    }
}
```

```
void RenderBanner()
{
    readonly bool isMacOs = platform.ToUpper().IndexOf("MAC") > -1;
    readonly bool isIE = browser.ToUpper().IndexOf("IE") > -1;
    readonly bool wasResized = resize > 0;

    if (isMacOs && isIE && wasInitialized() && wasResized)
    {
        // do something
    }
}
```

Introduce constant

Introduce constant

- Introduce constant for '40M'
- Introduce constant for all occurrences of '40M'
- Introduce local constant for '40M'
- Introduce local constant for all occurrences of '40M'

```
decimal checkedFee = numChecked * 40M;
```

```
Console.WriteLine($"Checked bag price: {checkedFee}");
total += checkedFee;

if (travelTime.Month >= 11 || travelTime.Month <= 1)
{
    Console.WriteLine($"Holiday Travel Fee: {total * HolidayTravelFeePercent}");
    total += total * HolidayTravelFeePercent;
}
```

Lines 4 to 5

```
{
    private const decimal V = 40M;

    public decimal HolidayTravelFeePercent { get; set; } = 0.15M;
}
```

Lines 22 to 24

```
{
    decimal checkedFee = numChecked * 40M;
    decimal checkedFee = numChecked * V;
    Console.WriteLine($"Checked bag price: {checkedFee}");
}
```

Lines 28 to 30

```
{
    decimal checkedFee = (numPassengers * 40M);
    decimal checkedFee = (numPassengers * V) + 10M;
    Console.WriteLine($"Checked bag price: {checkedFee}");
}
```

Preview changes

Extract method

```
void PrintOffer()
{
    PrintBanner();

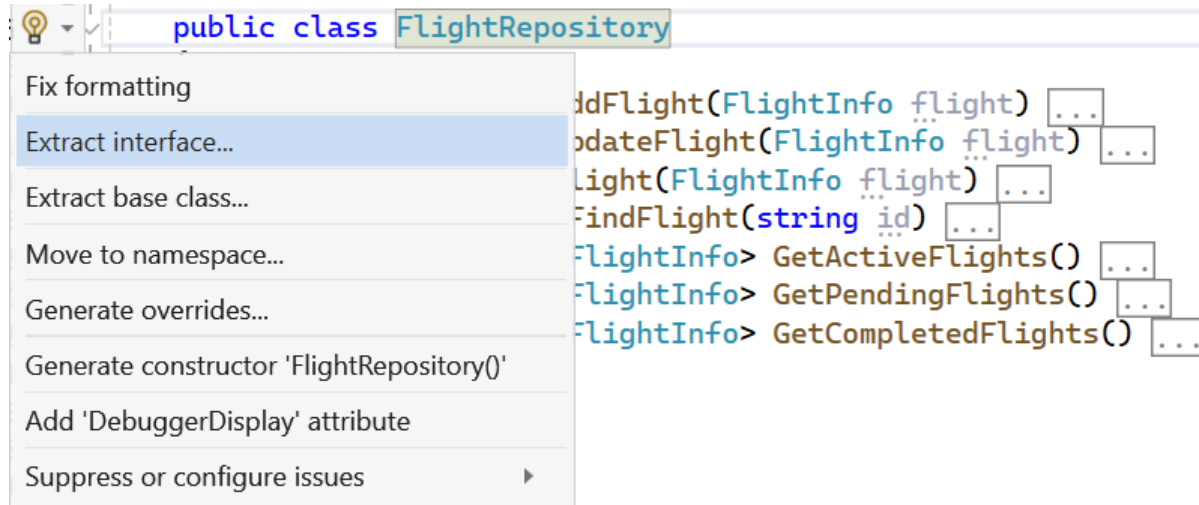
    // Print details.
    Console.WriteLine("Name: " + Name);
    Console.WriteLine("Amount: " + GetPrice());
}
```

```
void PrintOffer()
{
    PrintBanner();

    PrintDetails();
}

private void PrintDetails()
{
    Console.WriteLine("Name: " + Name);
    Console.WriteLine("Amount: " + GetPrice());
}
```

Extract interface



```
public interface IFlightRepository
{
    FlightInfo AddFlight(FlightInfo flight);
    void CancelFlight(FlightInfo flight);
    FlightInfo? FindFlight(string id);
    IEnumerable<FlightInfo> GetActiveFlights();
    IEnumerable<FlightInfo> GetCompletedFlights();
    IEnumerable<FlightInfo> GetPendingFlights();
    FlightInfo UpdateFlight(FlightInfo flight);
}

public class FlightRepository : IFlightRepository
{
    public FlightInfo AddFlight(FlightInfo flight) ...
    public FlightInfo UpdateFlight(FlightInfo flight) ...
    public void CancelFlight(FlightInfo flight) ...
    public FlightInfo? FindFlight(string id) ...
    public IEnumerable<FlightInfo> GetActiveFlights() ...
    public IEnumerable<FlightInfo> GetPendingFlights() ...
    public IEnumerable<FlightInfo> GetCompletedFlights() ...
}
```

Code smells

- It's difficult to understand what it does or why it does it
- You or people on your team avoid working with it
- It's slower to modify than other areas or tends to break when modified
- It's hard to test or debug
- Long method, large class, many parameters etc.

